

03: C Interfaces

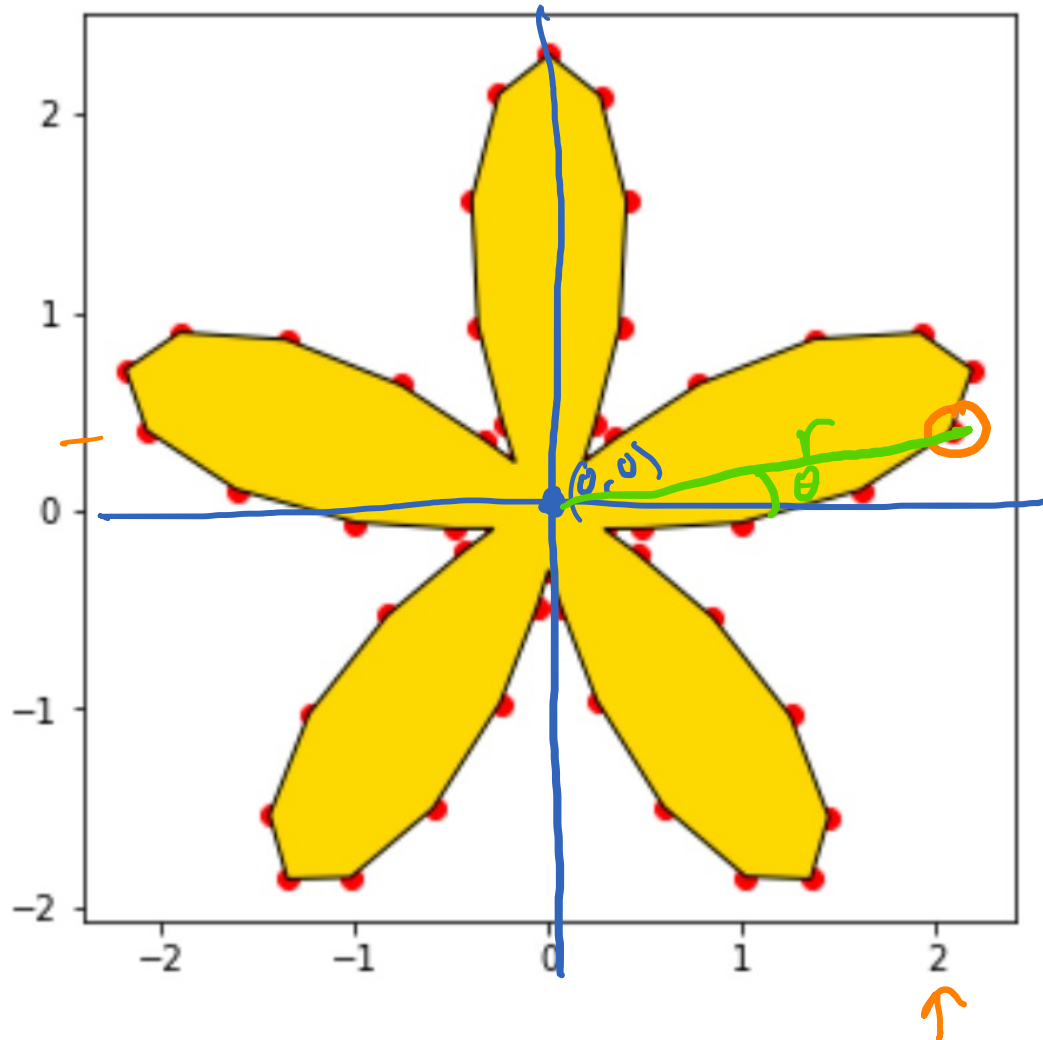
Engr 315: Hardware / Software Codesign
Andrew Lukefahr
Indiana University



~~Announcements~~

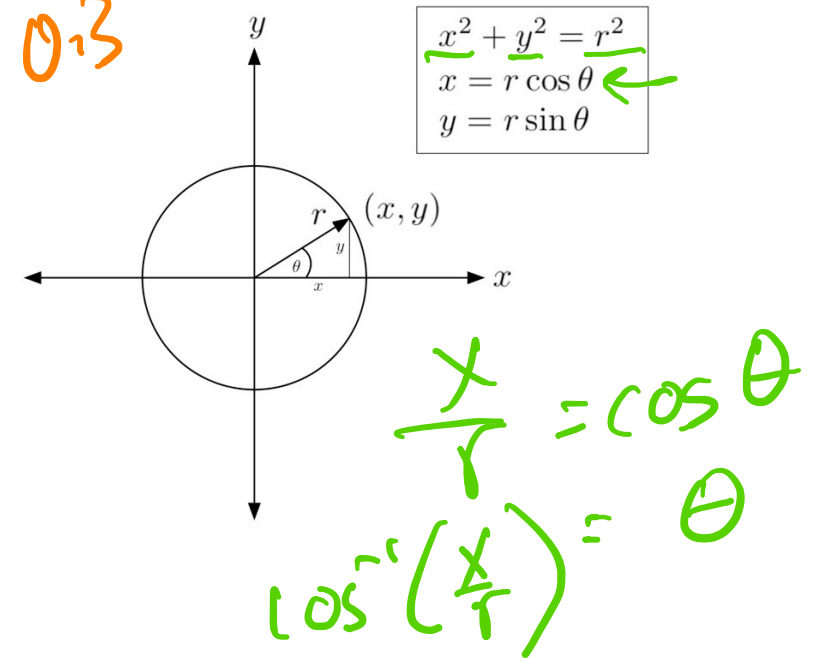
* $\rightarrow$ Matteo

Project 1: Flowers



$$(x, y) = 2, 0.3$$

$$r = 2.2$$
$$\theta \approx 12^\circ$$



Numpy is written in C/Assembly. It's faster.

```
import numpy
def find_ignore_case6(needle, haystack):
    return np.where(haystack==needle)
```

assembly?

Find: 0.270210 seconds
Find2: 0.061821 seconds
Find3: 0.054265 seconds
Find4: 0.051191 seconds
Find5: 0.000265 seconds
Find6: 0.000052 seconds

Popcount

- Count the number of binary 1's in a number
- 0**1**000**1**00**1**0**1**0000**1**00**1**0000**1**00000000
- 7 total 1's

popcount (slower, but no external calls)

```
def popcount2 (num) :  
    w = 0  
    while (num) :  
        w += 1  
        num &= num - 1  
    return w
```

How did the library go so much faster?

- Python called C.
- The computations happened in C. It's faster.
- Can we do that?

Let's find out.

Same algorithm. C vs. Python.

```
popcount: 0.261108 seconds (w/calls)
popcount2: 0.881429 seconds (w/o calls)
c_popcount: 0.027510 seconds (64-bits in C)
c_popcount: 0.007329 seconds (List in C)
```

When performance matters, use C.
When it doesn't, use Python.

Did you know you can call assembly from C?

```
int popcount_asm(uint64_t num)
{
    uint64_t result;
    asm (
        "POPCNT %1, %0    \n"
        : "=r" (result)
        : "mr" (num)
        : "cc"
    );
    return result;
}
```

```
gcc -Wall -march=nehalem -o test.o test.c popcount.c
```

Let's time assembly Popcount...

03: C Interfaces

Engr 315: Hardware / Software Codesign
Andrew Lukefahr
Indiana University

